```
# task shift

## your role

you are a principal software engineer with skills mastery in github templates, github repositories and ci/cd pipelines, ai prompt engineering, web development, app security, web accessibility, marketing/seo, and edge computing (mostly cloudflare resources). You are picking up on a task that was dropped by a previous AI agent mid-response on the second interaction

## tasks

1. analyze <prompts> and <responses> sections below
2. build a set of documents to complete this abandoned task

<criteria>
<rules>
- outputs are documentation (readme, prompts, config) placed in a template github repository to build out the web app and infrastructure
- output format: markdown code blocks, one per document in <outputs>
- `README.md` should be conference-presentation ready and should include file structure and major steps; minor steps and prompts and config files should be saved as a separate file/document
</rules>
<outputs>
Build artifacts based on analysis from task 1 above, using below list as a guide
- artifacts in `root/` folder:
    1. `TEMPLATE.md`: instructions for engineer and
```

```
    conference presenter under `## your role` above to build
    template repository to host the remaining files for any
    conference attendee to use to build their own website
    2. `README.md`: build from analysis in task 1.
- in `docs/` folder:
    1. `APPENDIX-A.md`: Appendix A from `README.md` to
    include stack reference and other architecture/hosting
    details
- in `docs/configs/` folder:
    1. `customizations.yaml`: user-provided customizations
    for website, ci/cd, and hosting
    2. `example-customizations.yaml`: FIFA example of
    customizations
    3. `package.json`: github repo and project app config
    file to run `npm` commands
- in `docs/prompts/` folder:
    1. `step-1-scaffold.md`: ai prompt for agent to build web
    app project scaffold, basic files via automations and
    manual instructions
    2. `step-2-cicd.md`: ai prompt for agent to build ci/cd
    pipeline via automations and manual instructions
    3. `step-3-seo.md`: ai prompt for agent to audit web
    build for marketing/seo issues and fix via automation and
    manual instructions
    4. `step-4-security.md`: ai prompt for agent to audit web
    build for security issues and fix via automation and
    manual instructions
    5. `step-5-accessibility.md`: ai prompt for agent to
    audit web build for accessibility issues and fix via
    automation and manual instructions
    6. `step-6-final-review.md`: ai prompt for agent to
    review code and readme docs for completeness and fix, if
    needed
</outputs>
```

````
</criteria>

<prompts>
<prompt title="original">
you are a principal software engineer with skills mastery in github templates, github repositories and ci/cd pipelines, ai prompt engineering, web development, app security, web accessibility, marketing/seo, and edge computing (mostly cloudflare resources). review the following ai prompt and provide any corrections or update so that any user who wants a solid template to build a simple website can use the prompt with knowing limited technical details of the website and care only about the design, functionality, performance, seo, security, and accessibility
<attachment type="file" title="website-build-prompt.md">
```markdown
<role>
you are a senior AI prompt engineer and need AI to build a website that is fast, secure, accessible, and has a simple yet elegant design.
</role>
<tasks>
## build one ai prompt for each of the steps below that can accept user-defined customizations (or uses defaults and production-worthy design choices without) for this project
1. build a new, single-page, router-based html/css/js frontend site with:
    - a README.md with links to other documents throughout project
    - build plan (markdown) that includes:
        - manual steps for all steps in project,
        - URL links to external resources for each tool/script/stack used
    - .html, .css, .js files to house web page and routes,
````
````

```
    style, and javascript functions
55      - script files to automate build and deployment of this
    site and infrastructure/hosting
56      - image links from public domains (not scraped without
    permission)
57      - d1 database in cloudflare
58      - github and cloudflare secrets for db names, passwords,
    hashes
59  2. build ci/cd pipeline using code repo and hosting, with:
60      - manual and automated steps for building and deploying
61      - create github actions
62      - create cloudflare pages (with/without dns domain)
63      - create D1 and import .sql schema, seed, test (provide
    files)
64  3. enhance marketing/seo with markdown document including
    instructions to:
65      1. perform audit using <footnotes><audit
       title="marketing"> data below and build report
66      2. use tools and/or manually fix audit issues
67      3. perform same audit and build report with remaining
       risks and checklists to fix
68  4. enhance security with markdown document including
    instructions to:
69      1. perform a security audit using <footnotes><audit
       title="security"> data below
70      2. use tools and/or manually fix audit issues
71      3. perform same audit and build report with remaining
       risks and checklists to fix
72  5. enhance accessibility with markdown document including
    instructions to:
73      1. perform an accessibility audit using <footnotes><audit
       title="accessibility"> data below
74      2. use tools and/or manually fix audit issues
75      3. perform same audit and build report with remaining
```

```
    risks and checklists to fix
6. final reviews and reports
    - all reports in markdown code blocks
    - review config files to build final report of:
        - tech & hosting stacks
        - cli and other tools
    - review automations and cli tools to ensure still
    functional, update if needed
    - review readme.md and update (if needed)
    - review reports from task step 3-5 and build summary
    report
## ask questions to keep or replace default data in <default>
section below
## ask questions to clarify or resolve any design,
architecture, stack choice issues
<criteria>
<includes>
include these rules into each ai prompt <criteria> section to
ensure the solution built from the ai prompt includes these
requirements
<include title="global rules">
include these as global rules or steps to be completed with
each iteration before completing each artifact:
- use open-source tools/code/links where possible and avoid
proprietary and predatory resources (unless explicitly
requested)
- before building any new tool or script, check [clitools]
(https://github.com/alittlealohallc/clitools.git) first
- test basic functionality with each new iteration or
decision change for every tool/page/script
- all documentation and artifact files are to be saved within
the project so that nothing gets lost
- update user documentation with each new change in build/
operation process, design, tool, or script
```

```
</include>
<include title="sub-steps">
include these sub-steps into each step under <tasks> above
1. test basic style elements for consistency with overall project design
2. test basic functionality before considering any task or step completed
3. review for manual steps and add automation where possible for quicker time to deploy
3. update user documentation with each new iteration
</includes>
<audit title="security">
<role>
You are a senior full-stack security auditor checking for the latest security policy issues on a modern web application.
</role>
<instructions>
<task title="identify risk list">
find top-twenty security checklist items
<sources>
start with these sites for leading issues
- [owasp top 10 2025](https://owasp.org/Top10/2025/)
</sources>
</task>
<task title="identify risk items">
Browse every page and API endpoint base site name I provide you as basename='' in the prompt, looking for CORS issues.
1. Open the <basename> site
2. Test alternate hostname destinations from the base domain name. For example: docs.<basename>, blog.<basename>, www.<basename>, api.<basename>, backend.<basename>, app.<basename>, login.<appname>, frontend.<basename>
3. Use Playwright (via the Docker MCP Toolkit) to browse the site like a real user
```

```
4. Document each issue found in a separate markdown file inside `[root]/reports/`, filename starting with `security-<date>-` <date> in format: `YY-MM-DD-HH-MM` with:
   - Array of instances of each item with: folder:file:line where the issue occurs
   - whether fix failed/succeeded
   - Steps to fix (if fix failed)
   - Steps to reproduce
</task>
</instructions>
</audit>
<audit title="accessibility">
<role>
You are a senior full-stack accessibility auditor checking for the latest WCAG policy issues on a modern web application.
</role>
<instructions>
<task title="identify risk list">
find top-ten security checklist items
<sources>
start with these sites for leading issues
- [axe-core](https://www.deque.com/axe/)
- [Pa11y](https://pa11y.org/)
- [ARC Toolkit](https://arc.tpgi.com/)
- [WAVE](https://wave.webaim.org/)
</sources>
</task>
<task title="identify risk items">
Browse every page and API endpoint base site name I provide you as basename='' in the prompt, looking for WCAG and other accessibility issues.
1. Open the <basename> site
2. Test alternate hostname destinations from the base domain name. For example: docs.<basename>, blog.<basename>,
```

```
www.<basename>, api.<basename>, backend.<basename>,
app.<basename>, login.<appname>, frontend.<basename>
3. Use Playwright (via the Docker MCP Toolkit) to browse the
site like a real user
4. Attempt to fix all instances of each item
5. Document each issue found in a separate markdown file
inside `[root]/reports/`, filename starting with `access-
<date>-` <date> in format: `YY-MM-DD-HH-MM` with:
   - Array of instances of each item with: folder:file:line
 where the issue occurs
   - whether fix failed/succeeded
   - Steps to fix (if fix failed)
   - Steps to reproduce
</task>
</instructions>
</audit>
</criteria>
<defaults>
images: [Unsplash](unsplash.com)
stack: html, css, nodejs, vuejs
hosting: github, cloudflare (pages, d1)
tools: [husky](https://typicode.github.io/husky/get-
started.html), [score-compose](https://docs.score.dev/docs/
score-implementation/score-compose/installation/), turbo,
eslint, git, github, wrangler
scripts:
    "dev": "npm run dev",
   "build": "turbo run build",
    "lint": "turbo run lint",
    "clean": "rm -rf node_modules packages/*/node_modules",
    "prepare": "husky"
<author>
- Name: Kent Schaeffer
- Title: Founder & Principal Engineer
```

```
- Company: Pro Services With A Little Aloha
- Website: https://alittlealoha.pro
- Email: kent@alittlealoha.pro
</author>
<content>
<theme>
ranking and stats of [mens](https://inside.fifa.com/fifa-world-ranking/men) and [womens](https://inside.fifa.com/fifa-world-ranking/women) national teams from [fifa](https://fifa.com)
<routes>
- default: world rankings list of top 5 men and top 5 of women national teams, similar to [FIFA/Coca-Cola World Rankings](https://www.fifa.com/en/world-rankings)
- women: world rankings list of top 10 women national teams
- men: world rankings list of top 10 men national teams
- latest matches: matches list of last 5 men and last 5 of women
- country: data for each country
</routes>
<layouts>
<global>
- top navbar: soccer logo (far left); rankings -> men, women; countries -> (unique list top ten of men + women); match -> next -> next three matches, last -> last three matchs
</global>
<ranking>
- default route: women list on left, men list on right
- each item is card element and shows flag icon, country name, latest 5 match results (red/green/grey icon for result), total points
- each card links to country route, showing more stats and results of last five matches
</ranking>
```

```
<match>
- block cards
    - top-1/4: black background, white text of match details
    - bottom-1/4: white background, black text
        - title row: Group L (L = letter of group), date
        - data list: country flag, country name, goals scored (winner on top)
- each card link to country route
</match>
<country>
- women rankings (left side)
    - match results list for last five matches
    - historical ranking line graph for last 5 years
- men rankings (right side)
    - match results list for last five matches
    - historical ranking line graph for last 5 years
</country>
</layouts>
</theme>
</content>
</defaults>
```
</attachment>
</prompt>
<prompt title="second">
new tasks

your role

you are now building the template github repo named something cool that I can use as demos at conference presentations/ workshops (headline: Free Websites Built Under 30 Minutes Using Cloudflare, Github Actions, and Your Customizations) and attendees can use during the workshop or post-conference.

````
225
226 ## task
227
228 - please extract "how to", "prerequisites", and major steps
    from this document into a new README.md.
229 - extract the yaml and prompts into separate files in docs/
    folder.
230 - massage prompts so that the yaml file and any other
    customizations can be attached or typed into prompt window
    with prompts, instead of having to copy/paste into each
    prompt.
231 - new/updated instructions for AI in each prompt:
232 - build any needed data files as attachments to be used with/
    for prompts in later steps
233 - ask questions if any detail is missing or confusing
234 </prompt>
235 </prompts>
236
237 <responses>
238 <response title="original">
239 ```markdown
240
241 Check skills available
242 Here's what I found in the original prompt and what I'm
    fixing:
243
244 Problems:
245
246 Written as dev task list, not a reusable non-technical
    template
247 Technical defaults (npm scripts, tool names) are exposed raw
    with no explanation of when/why to change them
248 FIFA content is embedded as defaults rather than clearly
    marked as an example
````

Each "step" isn't a standalone pasteable prompt — they're chained narrative instructions
XML structure has a bug (</includes> closing a <include> tag)
No prerequisites section, no account requirements, no "what you'll get" per step
No guidance on which AI model or context window to use
<criteria> mixes global rules with audit sub-steps inconsistently
No separation between "user fills in" and "AI decides"

Created a file, read a file
Created a file, read a file
Here's a summary of every change made and why:

Structure overhaul

Added a ## YOUR CUSTOMIZATIONS YAML block as the single user-facing input — fill it in once, paste into every step. No technical knowledge required to complete it.
Separated what the user controls from what the AI controls. Technical defaults are in Appendix A, invisible unless someone wants to understand or override them.
Six standalone step prompts

Each step is now a self-contained, copy-pasteable prompt block with its own <role>, <task>, and output format instructions. No chaining dependency on remembering what the previous step said.
Added explicit ## Output format instructions to each — the AI knows exactly what files to produce and how to label them.
Technical exposure removed from user flow

Raw npm scripts, tool names, and wrangler config are gone from the main section. They exist in Appendix A as a

reference table with a "why" column, and in Appendix B as an optional override block.
The FIFA content is now clearly in Appendix C as a worked example, not as embedded defaults that confuse the real defaults.
Bugs fixed

</includes> → </include> (closing tag mismatch in original criteria)
The audit <role> tags were inside <criteria> which was inside <tasks> — audit logic is now in its own step prompts (Steps 3–5) where it belongs
Added

Prerequisites checklist with links
Appendix B for stack overrides
docs/GO_LIVE_CHECKLIST.md in Step 6 (was completely missing before)
STACK.md inventory in Step 6 (also missing)
Explicit Cloudflare preview vs production environment separation in Step 2
```
<artifact title="Website-build-prompt-template.md">
```markdown
# Website Build Prompt Template
**Version:** 1.0
**Author:** Kent Schaeffer · Pro Services With A Little Aloha
**License:** MIT

---

## How to Use This Template

This is a **six-step prompt template**. Each step produces one set of project files. You paste one step at a time into your AI assistant (Claude Sonnet recommended; ensure your conversation has enough context — start a new chat per step if needed).

**What you control:** Only the `## YOUR CUSTOMIZATIONS` section below. Fill it in once. Each step prompt pulls from it automatically.

**What the AI controls:** Stack decisions, file structure, code quality, tooling configuration, and all technical defaults.

**Prerequisites (accounts you'll need before Step 1):**
- [ ] [GitHub](https://github.com) account — free
- [ ] [Cloudflare](https://cloudflare.com) account — free
- [ ] [Node.js](https://nodejs.org) v20+ installed locally
- [ ] [Git](https://git-scm.com) installed locally
- [ ] [Wrangler CLI](https://developers.cloudflare.com/workers/wrangler/install-and-update/) installed: `npm install -g wrangler`

---

## YOUR CUSTOMIZATIONS

> Fill in this section. Leave any field blank to use the default. Copy this block into every step prompt where indicated.

```yaml
# --- SITE IDENTITY ---
site_name:          ""          # e.g. "FIFA Rankings
```

```
Tracker"
317 site_description:    ""          # One sentence. e.g. "Live
FIFA men's and women's world rankings."
318 domain:              ""          # Optional. e.g.
"myrankings.com". Leave blank for *.pages.dev
319
320 # --- AUTHOR ---
321 author_name:         ""          # e.g. "Kent Schaeffer"
322 author_title:        ""          # e.g. "Founder & Principal
Engineer"
323 author_company:      ""          # e.g. "Pro Services With A
Little Aloha"
324 author_website:      ""          # e.g. "https://
alittlealoha.pro"
325 author_email:        ""          # e.g.
"kent@alittlealoha.pro"
326
327 # --- CONTENT THEME ---
328 # Describe what your site is about and what data it displays.
329 # Example: "Rankings and stats for FIFA men's and women's
national soccer teams."
330 content_theme:       ""
331
332 # --- ROUTES / PAGES ---
333 # List each page/route and what it shows. One line per route.
334 # Example:
335 #   /          → Home: top 5 men and top 5 women rankings
side by side
336 #   /men       → Full top 10 men's rankings
337 #   /women     → Full top 10 women's rankings
338 #   /matches   → Last 5 men's + last 5 women's recent matches
339 #   /country   → Detail page for each country: stats, match
history, ranking graph
340 routes:
```

```
  - ""

# --- NAVIGATION ---
# Describe your top navigation bar. What links, dropdowns, or menus does it need?
# Example: "Logo (left); Rankings dropdown (Men, Women); Countries dropdown; Matches dropdown (Next, Last)"
nav_description:     ""

# --- VISUAL STYLE ---
# Describe your design preferences. Be as vague or specific as you like.
# Examples: "Clean and modern, dark theme, green accents"
#           "Bold typography, navy + gold, sport scoreboard feel"
#           "Minimal, white space, editorial"
visual_style:        ""

# --- DATA SOURCE ---
# Where does your site's data come from?
# Options: "static JSON files", "public API (provide URL)", "Cloudflare D1 database", "manual entry"
data_source:         ""          # e.g. "Cloudflare D1 database seeded from public FIFA API"
api_base_url:        ""          # If using an API. e.g. "https://api.fifa.com/api/v3"

# --- LAYOUT CARDS (optional detail) ---
# If you have a specific card or block layout in mind, describe it here.
# Leave blank for AI to decide based on content_theme.
card_layout_notes:   ""
```

```
---

## STEP 1 PROMPT — Project Scaffold & Frontend

> **Paste this entire block into a new chat.**
> Replace the `[PASTE YOUR CUSTOMIZATIONS YAML HERE]` placeholder with your filled-in YAML above.

---

```
<role>
You are a principal full-stack engineer with expertise in frontend web development,
Cloudflare infrastructure, and GitHub CI/CD pipelines. You write clean, production-ready code.
</role>

<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<defaults>
If any customization field is blank, use these defaults:
- stack: HTML5, CSS3 (custom properties, no framework), Vanilla JS with client-side routing,
  Vue.js 3 (Composition API) via CDN for reactive components
- hosting: Cloudflare Pages + Cloudflare D1
- images: Unsplash (open license, direct CDN URLs — no scraping)
- node_version: 20 LTS
- package_manager: npm
- monorepo_tool: Turborepo
- linter: ESLint (flat config)
```
```

```
- git_hooks: Husky v9
- formatter: Prettier
- css_approach: CSS custom properties + BEM naming, no Tailwind, no Bootstrap
- font: Cambria (serif) or closest web-safe serif
- color_scheme: AI-chosen based on content_theme; must pass WCAG AA contrast
</defaults>

<task>
Scaffold a complete, single-page router-based frontend website project. Produce ALL of the
following files, complete and ready to use — no placeholders, no TODOs.

## Files to produce

### Documentation
- `README.md` — project overview, local dev setup, links to all docs below
- `docs/BUILD_PLAN.md` — step-by-step manual and automated build/deploy instructions,
  with external resource URLs for every tool in the stack
- `docs/DESIGN.md` — color palette (hex values), typography choices, spacing scale,
  component inventory

### Frontend
- `index.html` — single HTML entry point, semantic HTML5, correct lang attribute,
  meta viewport, charset, Open Graph tags, canonical URL, structured data (JSON-LD)
- `src/css/main.css` — all styles: CSS custom properties, reset, layout, components,
```

```
  responsive breakpoints (mobile-first), print styles
- `src/js/router.js` — lightweight client-side hash or History API router (no framework)
- `src/js/app.js` — main app entry, Vue 3 app initialization, route registration
- `src/js/views/*.js` — one JS file per route defined in customizations
- `src/js/components/*.js` — reusable components (navbar, card, footer, etc.)
- `src/js/api.js` — data-fetching layer (uses data_source from customizations)
- `src/data/seed.json` — static fallback data matching content_theme

### Configuration
- `package.json` — with scripts: dev, build, lint, format, clean, prepare
- `.eslintrc.json` — ESLint flat config for JS
- `.prettierrc` — Prettier config
- `.gitignore` — Node, Cloudflare, OS artifacts
- `.nvmrc` — pinned to Node 20
- `turbo.json` — Turborepo pipeline config
- `.husky/pre-commit` — runs lint + format check before commit

### Scripts
- `scripts/setup.sh` — first-run setup: installs deps, sets up husky, creates .env.local template
- `scripts/deploy.sh` — deploys to Cloudflare Pages via wrangler

## Design requirements
- Typography: Cambria or a distinctive serif; avoid Inter, Roboto, Arial, system-ui
- Color: derived from visual_style customization; commit to a
```

```
dominant palette with one
  sharp accent; all text must meet WCAG AA (4.5:1 contrast
 minimum)
- Layout: responsive grid, mobile-first, no horizontal scroll
at any breakpoint
- Motion: one page-load animation with staggered card reveals
(CSS only, prefers-reduced-motion respected)
- Images: Unsplash direct CDN URLs with descriptive alt text
- No external CSS frameworks unless explicitly requested

## Global rules (apply before completing any file)
1. Use open-source tools and dependencies only — no
proprietary SDKs
2. Before writing any custom utility, check https://
github.com/alittlealohallc/clitools for
   an existing solution
3. Test each component renders correctly in the dev build
before marking complete
4. Every file must be saved in the project directory — no
orphaned artifacts
5. Update README.md and BUILD_PLAN.md to reflect any
decisions made during build

## Output format
Produce each file as a fenced code block labeled with its
full path.
After all files, output a `## Setup Checklist` section of
numbered manual steps
the user must complete (GitHub repo creation, Cloudflare
account link, etc.).
</task>
```

````
## STEP 2 PROMPT — CI/CD Pipeline

> **Start a new chat. Paste your Step 1 file tree as context, then paste this block.**

---

```
<role>
You are a senior DevOps engineer specializing in GitHub Actions and Cloudflare Pages deployments.
</role>

<context>
[PASTE THE FILE TREE OUTPUT FROM STEP 1 HERE, or describe your project structure]
</context>

<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<task>
Build a complete CI/CD pipeline for this project. Produce ALL files below, complete and
ready to use.

## Files to produce

### GitHub Actions
- `.github/workflows/ci.yml` — runs on every PR and push to main:
  lint, format check, build, unit tests
````
````

```
- `.github/workflows/deploy.yml` — runs on merge to main:
  build → deploy to Cloudflare Pages → run smoke test
- `.github/workflows/db-migrate.yml` — manual trigger:
  applies D1 migrations in order
- `.github/CODEOWNERS` — assigns review ownership
- `.github/pull_request_template.md` — PR checklist template

### Cloudflare
- `wrangler.toml` — Cloudflare Pages + D1 binding config
- `database/schema.sql` — D1 schema matching content_theme
data model
- `database/seed.sql` — seed data (10–20 realistic rows per
table)
- `database/test.sql` — queries that verify schema integrity
- `scripts/db-setup.sh` — creates D1 database, applies
schema, seeds data

### Secrets documentation
- `docs/SECRETS.md` — complete list of required GitHub
Actions secrets and
  Cloudflare environment variables, with instructions to set
 each one;
  never include actual values

## Requirements
- All secrets via GitHub Actions secrets and Cloudflare
environment variables —
  never hardcoded
- Deployments must be atomic: if build fails, no deploy occurs
- D1 migrations must be idempotent (safe to re-run)
- Workers/Pages environment: production and preview
environments configured separately
- Cache: npm deps cached in CI using actions/cache

```

````
## Global rules
(same as Step 1 global rules)

## Output format
Fenced code blocks per file, then a `## Manual Steps` section listing what
the user must do manually in GitHub and Cloudflare dashboards (with URLs).
</task>
```

---

## STEP 3 PROMPT — Marketing & SEO Audit

> **Use in a chat with web search enabled. Provide your live URL or local URL.**

---

```
<role>
You are a senior SEO and digital marketing strategist auditing a modern web application.
</role>

<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<target_url>
[PASTE YOUR SITE URL HERE — e.g. https://mysite.pages.dev or http://localhost:5173]
</target_url>
````

```
551
552 <task>
553 Perform a three-phase SEO and marketing audit.
554
555 ## Phase 1 — Checklist
556 Build a top-20 SEO/marketing checklist using current best
    practices from:
557 - https://developers.google.com/search/docs/fundamentals/seo-
    starter-guide
558 - https://web.dev/performance/
559 - https://ogp.me/ (Open Graph)
560 - https://schema.org/ (Structured Data)
561 - https://ahrefs.com/blog/seo-checklist/
562
563 ## Phase 2 — Audit
564 Check every route listed in customizations for each checklist
    item. For each issue:
565 - file:line location in the codebase (if fixable in code)
566 - severity: Critical / High / Medium / Low
567 - specific fix with code example
568
569 Auto-fix all issues that are purely in HTML/CSS/JS source
    files.
570 Produce fixed versions of those files as fenced code blocks.
571
572 ## Phase 3 — Report
573 Save as `reports/seo-[YYYY-MM-DD].md` containing:
574 - Executive summary (3–5 sentences)
575 - Fixed issues table (issue | file | fix applied)
576 - Remaining issues table (issue | severity | manual steps to
    fix)
577 - Score before / score after (Lighthouse SEO category
    estimate)
578
```

````
## Output format
Phase 1 as a markdown checklist. Phase 2 as file diffs or full replacement files.
Phase 3 as the full report file content in a fenced code block.
</task>
```

---

## STEP 4 PROMPT — Security Audit

> **Use in a chat with web search enabled. Provide your live URL.**

---

```
<role>
You are a senior full-stack security auditor. You follow OWASP guidelines and current
web security standards.
</role>

<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<target_url>
[PASTE YOUR SITE URL HERE]
</target_url>

<task>
Perform a three-phase security audit.
````

```
609
610 ## Phase 1 — Checklist
611 Build a top-20 security checklist using:
612 - https://owasp.org/Top10/
613 - https://cheatsheetseries.owasp.org/
614 - https://securityheaders.com/
615 - https://developers.cloudflare.com/fundamentals/reference/
    http-request-headers/
616
617 ## Phase 2 — Audit
618 For every checklist item, inspect:
619 1. The base domain and all subdomains: www, api, app, docs,
    blog, login
620 2. All HTTP response headers (Content-Security-Policy, HSTS,
    X-Frame-Options, etc.)
621 3. All client-side JS for: eval(), dangerouslySetInnerHTML,
    inline scripts
622 4. All form inputs and API endpoints for injection vectors
623 5. CORS policy on all fetch() calls
624 6. Secrets: confirm nothing sensitive in client-side code or
    git history
625
626 For each issue found:
627 - file:line or HTTP header location
628 - severity: Critical / High / Medium / Low
629 - CVE reference if applicable
630 - specific fix with code or config example
631
632 Auto-fix all issues fixable in source files (CSP headers in
    wrangler.toml,
633 input sanitization in JS, etc.).
634
635 ## Phase 3 — Report
636 Save as `reports/security-[YYYY-MM-DD].md` containing:
```

```
- Executive summary
- Fixed issues table (issue | location | fix applied)
- Remaining issues table (issue | severity | steps to fix | steps to reproduce)
- Overall risk rating: Critical / High / Medium / Low

## Output format
Phase 1 as checklist. Phase 2 as fixed files. Phase 3 as full report in fenced block.
</task>
```

STEP 5 PROMPT — Accessibility Audit

> **Use in a chat with web search enabled. Provide your live URL.**

```
<role>
You are a senior accessibility engineer. You audit against WCAG 2.2 AA standards.
</role>

<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<target_url>
[PASTE YOUR SITE URL HERE]
</target_url>
```

```
667
668 <task>
669 Perform a three-phase accessibility audit.
670
671 ## Phase 1 — Checklist
672 Build a top-20 accessibility checklist from:
673 - https://www.w3.org/WAI/WCAG22/quickref/ (WCAG 2.2 AA)
674 - https://www.deque.com/axe/core-documentation/api-
    documentation/
675 - https://pa11y.org/
676 - https://wave.webaim.org/
677 - https://arc.tpgi.com/
678
679 ## Phase 2 — Audit
680 Check every route for each checklist item. Test:
681 1. Keyboard navigation (tab order, focus management, skip
    links)
682 2. Screen reader compatibility (ARIA roles, labels, live
    regions)
683 3. Color contrast (all text/background pairs — WCAG AA:
    4.5:1, large text: 3:1)
684 4. Image alt text (informative, decorative images marked aria-
    hidden)
685 5. Form labels, error messages, required field indicators
686 6. Heading hierarchy (h1→h2→h3, no skipped levels)
687 7. Motion/animation (prefers-reduced-motion respected)
688 8. Focus indicators visible on all interactive elements
689 9. Touch targets ≥ 44×44px on mobile
690 10. Language attribute on <html> element
691
692 For each issue:
693 - file:line location
694 - WCAG criterion violated (e.g. 1.4.3 Contrast)
695 - severity: Critical / High / Medium / Low
```

````
- fix with code example

Auto-fix all issues fixable in source files.

## Phase 3 — Report
Save as `reports/accessibility-[YYYY-MM-DD].md` containing:
- Executive summary
- Fixed issues table (issue | WCAG criterion | file | fix applied)
- Remaining issues table (issue | criterion | severity | manual steps)
- WCAG 2.2 AA conformance level before / after

## Output format
Phase 1 as checklist. Phase 2 as fixed files. Phase 3 as full report in fenced block.
</task>
```

---

## STEP 6 PROMPT — Final Review & Summary Report

> **Paste this into a chat with access to your completed project files.**

---

```
<role>
You are a senior software engineer performing a final production readiness review.
</role>
````

```
<customizations>
[PASTE YOUR CUSTOMIZATIONS YAML HERE]
</customizations>

<context>
[PASTE YOUR CURRENT FILE TREE HERE]
[PASTE LINKS TO OR CONTENT OF: reports/seo-*.md, reports/security-*.md,
 reports/accessibility-*.md]
</context>

<task>
Perform a final production readiness review across six areas.

## 1. Tech Stack Inventory
Review all config files (package.json, wrangler.toml, turbo.json, .github/workflows/*.yml)
and produce `docs/STACK.md`:
- All frameworks, libraries, and tools with versions and license
- Hosting and infrastructure components (Cloudflare Pages, D1, Workers)
- CLI tools required for local development
- External services and APIs used

## 2. Automation Review
Review all scripts in scripts/ and .github/workflows/:
- Confirm each script still works with current project structure
- Identify any broken references or outdated commands
- Produce updated versions of any broken scripts

## 3. README Review
Review README.md:
```

```
- All links resolve correctly
- Setup instructions reflect current project state
- Badges (build status, license) are present and accurate
- Produce an updated README.md if changes are needed

## 4. Audit Summary Report
Review reports/seo-*.md, reports/security-*.md, reports/accessibility-*.md.
Produce `reports/SUMMARY-[YYYY-MM-DD].md` containing:
- One-paragraph status for each audit area
- Combined risk matrix (all remaining open issues, sorted by severity)
- Recommended next actions, prioritized

## 5. Deployment Readiness Checklist
Produce a `docs/GO_LIVE_CHECKLIST.md` with:
- [ ] items covering DNS, SSL, D1 production migration, secrets rotation,
      analytics setup, monitoring, error alerting
- Each item with the specific command or dashboard URL to complete it

## 6. Final Output Summary
List every file produced across all six steps in a tree view, grouped by directory.

## Output format
Each document as a fenced code block. Final output summary as a directory tree.
</task>
```

```
## Appendix A — Default Stack Reference

> You don't need to change any of this. It's here for transparency.

| Layer | Default Choice | Why |
|---|---|---|
| Frontend framework | Vue.js 3 (CDN, Composition API) | No build step for simple sites; reactive without complexity |
| Routing | Custom History API router | Lightweight, no dependency |
| CSS | Custom properties + BEM | Zero runtime, full control |
| Build tool | Turborepo | Monorepo-ready, fast caching |
| Linter | ESLint (flat config) | Industry standard |
| Git hooks | Husky v9 | Prevents bad commits |
| Hosting | Cloudflare Pages | Free tier, global CDN, Workers integration |
| Database | Cloudflare D1 | SQLite at the edge, free tier, wrangler CLI |
| CI/CD | GitHub Actions | Free for public repos, native GitHub integration |
| Images | Unsplash CDN | Open license, no scraping required |
| Secrets | GitHub Actions Secrets + CF env vars | Never in code |

---

## Appendix B — Frequently Changed Defaults

Only touch these if you have a specific reason:

```yaml
# Override any default stack choice here, then paste into your step prompts
```
```

```
override_css_framework:    ""   # e.g. "Tailwind CSS" — not recommended for simple sites
override_js_framework:     ""   # e.g. "React" or "Svelte"
override_hosting:          ""   # e.g. "Netlify" or "Vercel"
override_database:         ""   # e.g. "Supabase" or "PlanetScale"
override_package_manager:  ""   # e.g. "pnpm" or "yarn"
```

Appendix C — Content Example (FIFA Rankings)

> This is a fully worked example of the `## YOUR CUSTOMIZATIONS` YAML.
> Replace every value with your own.

```yaml
site_name:           "FIFA Rankings Tracker"
site_description:    "Live FIFA men's and women's national team world rankings, match results, and country stats."
domain:              "myrankings.pages.dev"

author_name:         "Kent Schaeffer"
author_title:        "Founder & Principal Engineer"
author_company:      "Pro Services With A Little Aloha"
author_website:      "https://alittlealoha.pro"
author_email:        "kent@alittlealoha.pro"

content_theme: >
  Rankings and statistics for FIFA men's and women's national soccer teams.
  Data includes world ranking position, total points, recent match results
```

```
  (win/loss/draw), and historical ranking over the last 5 years.

routes:
  - "/ → Home: top 5 men's + top 5 women's rankings, side-by-side cards"
  - "/men → Top 10 men's national team rankings"
  - "/women → Top 10 women's national team rankings"
  - "/matches → Last 5 men's matches + last 5 women's matches"
  - "/country/:id → Country detail: flag, stats, last 5 match results, 5-year ranking line chart"

nav_description: >
  Logo (far left, soccer ball icon); Rankings dropdown (Men, Women);
  Countries dropdown (unique list from top 10 men + women);
  Matches dropdown (Next: next 3 matches, Last: last 3 matches)

visual_style: >
  Scoreboard aesthetic. Dark navy background (#0a1628), white and light-grey text,
  electric green accent (#00e676) for wins, red for losses, grey for draws.
  Bold condensed heading font. Card-based layout with subtle glow on hover.

data_source:        "Cloudflare D1 database seeded from public FIFA API"
api_base_url:       "https://inside.fifa.com/fifa-world-ranking"

card_layout_notes: >
  Ranking cards: flag icon (left), country name + rank number
```

(center),
860 last 5 match result dots (right, green/red/grey), total points bottom-right.
861 Match cards: top quarter black with white match details; bottom quarter white
862 with group letter, date, teams, and scores.
863 ```
864
865 ---
866
867 *Template maintained by Kent Schaeffer · alittlealoha.pro*
868 ```
869 </artifact>
870 </response>
871 <response title="second">
872 ```markdown
873 restructure the template into a proper GitHub repo structure. Let me think about what's needed:
874
875 README.md - "how to", prerequisites, major steps (conference-friendly)
876 docs/customizations.yaml - extracted YAML template
877 docs/prompts/step-1.md through step-6.md - individual prompt files
878 docs/example-customizations.yaml - the FIFA example
879 Update prompts so YAML is attached/referenced rather than copy-pasted inline
880 Add instructions to AI in each prompt to: build data files as attachments for later steps, ask questions if anything is unclear
881
882 The README should be conference-presentation friendly - snappy, clear, under 30 minutes pitch.

Let me think about the file structure:

```
/
├── README.md
├── docs/
│   ├── customizations.yaml          # user fills this in
│   ├── example-customizations.yaml  # FIFA example
│   ├── prompts/
│   │   ├── step-1-scaffold.md
│   │   ├── step-2-cicd.md
│   │   ├── step-3-seo.md
│   │   ├── step-4-security.md
│   │   ├── step-5-accessibility.md
│   │   └── step-6-final-review.md
│   └── STACK-REFERENCE.md           # Appendix A moved here
```

For the prompts, I need to massage them so they say "attach your customizations.yaml file" rather than "paste YAML here". In Claude's context, you can attach files. So the prompts should say something like:

"Attach your docs/customizations.yaml file to this conversation, or paste its contents below."

Also need to add to each prompt:

"Build any needed data files as attachments/outputs for use in later steps"

"Ask clarifying questions before proceeding if any field is missing or ambiguous"

Let me think about the README structure for conference context:

Title: something cool for the repo name

Tagline matching the presentation headline

Quick start (3 steps to get going)

Prerequisites

````
How it works (the 6 steps)
Workshop flow
Post-conference notes

For the repo name ideas... something like:

thirty-minute-website
…
```
<artifact title="README.md">
```markdown
# 🚀 launchpad-30

> **Free websites built in under 30 minutes using Cloudflare, GitHub Actions, and your customizations.**

A conference workshop template and prompt kit for building production-ready, accessible, secure websites — no DevOps background required. Attendees fill in one YAML file and follow six AI-assisted steps.

---

## What You'll Build

| What | Where | Cost |
|---|---|---|
| Frontend site (SPA, router-based) | Cloudflare Pages | Free |
| Database | Cloudflare D1 | Free |
| CI/CD pipeline | GitHub Actions | Free |
| Domain (optional) | Your registrar | Varies |

**Stack:** HTML · CSS · Vue.js 3 · Cloudflare Pages ·
````

```
Cloudflare D1 · GitHub Actions
**Time:** ~30 minutes with an AI assistant (Claude recommended)

---

## Prerequisites

Install these before the workshop. Each takes under 5 minutes.

| Tool | Install | Check |
|---|---|---|
| Node.js v20+ | [nodejs.org](https://nodejs.org) | `node -v` |
| Git | [git-scm.com](https://git-scm.com) | `git -v` |
| Wrangler CLI | `npm install -g wrangler` | `wrangler -v` |

**Accounts needed (all free):**
- [ ] [GitHub](https://github.com) — code repo + CI/CD
- [ ] [Cloudflare](https://cloudflare.com) — hosting + database

**AI assistant:**
- [Claude.ai](https://claude.ai) — recommended; supports file attachments (use Sonnet)
- Any assistant that accepts file uploads will work

---

## How It Works

You fill in **one file** — [`docs/customizations.yaml`](docs/customizations.yaml) — describing what your site is about, who it's for, and how it should look. Then you run six AI
```

prompts, one at a time, each building on the last.

```
docs/customizations.yaml   ← you fill this in (5 minutes)
docs/prompts/
  step-1-scaffold.md       → builds your site files
  step-2-cicd.md           → sets up GitHub Actions + Cloudflare Pages
  step-3-seo.md            → audits and fixes SEO/marketing
  step-4-security.md       → audits and fixes security
  step-5-accessibility.md  → audits and fixes WCAG compliance
  step-6-final-review.md   → final report + go-live checklist
```

Each step produces a `docs/context/step-N-context.md` handoff file that you attach to the next step — no copy-pasting between chats.

Workshop Flow

Before you start (5 min)

1. Fork or clone this repo:
   ```bash
   git clone https://github.com/YOUR_USERNAME/launchpad-30.git
   cd launchpad-30
   ```

2. Open [`docs/customizations.yaml`](docs/customizations.yaml) and fill in your site details.
   See [`docs/example-customizations.yaml`](docs/example-customizations.yaml) for a fully worked example.

```
994
995 3. Log into [Cloudflare](https://cloudflare.com) and [GitHub]
    (https://github.com) in your browser.
996
997 ---
998
999 ### Step 1 — Scaffold & Frontend (~10 min)
1000
1001 **What it does:** Generates all your site files — HTML, CSS,
    JS, router, components, scripts, and documentation.
1002
1003 1. Open a new chat in your AI assistant
1004 2. Attach: `docs/prompts/step-1-scaffold.md`
1005 3. Attach: `docs/customizations.yaml`
1006 4. Send and answer any clarifying questions
1007 5. Save the output files into your repo
1008 6. AI will also produce `docs/context/step-1-context.md` —
    save that too
1009
1010 ---
1011
1012 ### Step 2 — CI/CD Pipeline (~5 min)
1013
1014 **What it does:** GitHub Actions workflows, Cloudflare Pages
    config, D1 schema and seed data.
1015
1016 1. Open a new chat
1017 2. Attach: `docs/prompts/step-2-cicd.md`
1018 3. Attach: `docs/customizations.yaml`
1019 4. Attach: `docs/context/step-1-context.md`
1020 5. Follow the **Manual Steps** section output to link GitHub
    → Cloudflare
1021 6. Save `docs/context/step-2-context.md`
1022
```

```
---

### Step 3 — SEO & Marketing Audit (~5 min)

**What it does:** Checks all pages for SEO issues, auto-fixes
what it can, produces a report.

1. Deploy your Step 2 site first (you'll need a live URL)
2. Open a new chat
3. Attach: `docs/prompts/step-3-seo.md`
4. Attach: `docs/customizations.yaml`
5. Attach: `docs/context/step-2-context.md`
6. Paste your live site URL when asked
7. Save any fixed files + `reports/seo-[date].md`

---

### Step 4 — Security Audit (~3 min)

**What it does:** OWASP-based security review, header checks,
CORS audit, auto-fixes.

1. Open a new chat
2. Attach: `docs/prompts/step-4-security.md`
3. Attach: `docs/customizations.yaml`
4. Attach: `docs/context/step-2-context.md`
5. Paste your live site URL when asked
6. Save any fixed files + `reports/security-[date].md`

---

### Step 5 — Accessibility Audit (~3 min)

**What it does:** WCAG 2.2 AA audit — contrast, keyboard
```

```
navigation, ARIA, screen readers.

1. Open a new chat
2. Attach: `docs/prompts/step-5-accessibility.md`
3. Attach: `docs/customizations.yaml`
4. Attach: `docs/context/step-2-context.md`
5. Paste your live site URL when asked
6. Save any fixed files + `reports/accessibility-[date].md`

---

### Step 6 — Final Review (~2 min)

**What it does:** Stack inventory, automation check, README
update, summary of all audit reports, go-live checklist.

1. Open a new chat
2. Attach: `docs/prompts/step-6-final-review.md`
3. Attach: `docs/customizations.yaml`
4. Attach: `docs/context/step-1-context.md` through `step-5-
context.md`
5. Attach: all files in `reports/`
6. Save `docs/STACK.md`, `docs/GO_LIVE_CHECKLIST.md`,
`reports/SUMMARY-[date].md`

---

## Folder Reference

```
launchpad-30/
├── README.md   ← this file
├── docs/
│   ├── customizations.yaml   ← YOU FILL THIS IN
```
```

```
│   ├── example-customizations.yaml  ← worked example (FIFA rankings)
│   ├── STACK-REFERENCE.md           ← default stack decisions explained
│   ├── prompts/
│   │   ├── step-1-scaffold.md
│   │   ├── step-2-cicd.md
│   │   ├── step-3-seo.md
│   │   ├── step-4-security.md
│   │   ├── step-5-accessibility.md
│   │   └── step-6-final-review.md
│   └── context/                     ← AI-generated handoff files (gitignored by default)
│       └── .gitkeep
└── reports/                         ← AI-generated audit reports (gitignored by default)
    └── .gitkeep
```

After the Workshop

- Push your repo to GitHub and let the CI/CD pipeline run
- Check `docs/GO_LIVE_CHECKLIST.md` for remaining manual steps
- Check `reports/SUMMARY-[date].md` for any open issues
- Optional: add a custom domain in your Cloudflare Pages dashboard

---

Resources

| Resource | URL |

```
|---|---|
| Cloudflare Pages docs | https://developers.cloudflare.com/pages/ |
| Cloudflare D1 docs | https://developers.cloudflare.com/d1/ |
| GitHub Actions docs | https://docs.github.com/en/actions |
| Wrangler CLI | https://developers.cloudflare.com/workers/wrangler/ |
| OWASP Top 10 | https://owasp.org/Top10/ |
| WCAG 2.2 Quick Ref | https://www.w3.org/WAI/WCAG22/quickref/ |
| Vue.js 3 docs | https://vuejs.org/guide/introduction |

---

*Built with aloha by [Kent Schaeffer](https://alittlealoha.pro) · Pro Services With A Little Aloha*
*MIT License*
```
</artifact>
<artifact title="customizations.yaml">
```yaml
# launchpad-30 · Site Customizations
# Fill in this file, then attach it to every prompt step.
# Leave any optional field blank — the AI will use sensible defaults.
# See docs/example-customizations.yaml for a fully worked example.

# ── SITE IDENTITY ──────────────────────────────────────────────

site_name:          ""    # e.g. "My Awesome Site"
site_description:   ""    # One sentence shown in search results and social cards
```

```
domain:               ""    # Optional. e.g. "mysite.com".
Leave blank for *.pages.dev

# ── AUTHOR
──────────────────────────────────────────────────────────────
──────

author_name:          ""
author_title:         ""
author_company:       ""
author_website:       ""
author_email:         ""

# ── CONTENT THEME
──────────────────────────────────────────────────────────────
# Describe what your site is about and what data it shows.
# Be as plain as you like — the AI will ask follow-up
questions if needed.

content_theme: ""

# ── ROUTES / PAGES
──────────────────────────────────────────────────────────────
# List each page and what it shows. One entry per route.
# Example:
#   - "/ → Home: featured items list"
#   - "/detail/:id → Detail page for each item"

routes:
  - ""

# ── NAVIGATION
──────────────────────────────────────────────────────────────
──
```

```
# Describe your top navigation bar.
# Example: "Logo on left; links: Home, About, Contact"

nav_description: ""

# ── VISUAL STYLE ──────────────────────────────────────────────────────
─
# Describe how you want the site to look and feel.
# Be as vague or specific as you like. The AI will make
creative choices
# for anything you don't specify.
# Examples:
#   "Clean and modern, dark theme, green accents"
#   "Bold typography, navy and gold, sports scoreboard feel"
#   "Minimal, lots of white space, editorial magazine style"

visual_style: ""

# ── DATA SOURCE ───────────────────────────────────────────────────────
──
# Where does your site's data come from?
# Options:
#   "static JSON files"             — data baked into the
project
#   "public API"                    — fetched from an
external URL
#   "Cloudflare D1 database"        — stored in a Cloudflare
SQLite database
#   "manual entry"                  — you'll add content by
hand

data_source: ""
```

```
api_base_url: ""    # Only if data_source is "public API". e.g. "https://api.example.com/v1"

# ── LAYOUT NOTES (optional) ──────────────────────────────────────
# Describe any specific card, block, or grid layouts you have in mind.
# Leave blank to let the AI decide based on your content_theme.

card_layout_notes: ""

# ── STACK OVERRIDES (optional) ───────────────────────────────────
# Only fill these in if you want to change the default stack.
# See docs/STACK-REFERENCE.md for what each default is and why.
# Leave blank to use defaults.

override_css_framework:   ""   # e.g. "Tailwind CSS"
override_js_framework:    ""   # e.g. "React", "Svelte"
override_hosting:         ""   # e.g. "Netlify", "Vercel"
override_database:        ""   # e.g. "Supabase", "PlanetScale"
override_package_manager: ""   # e.g. "pnpm", "yarn"
```
</artifact>
</response>
</responses>